

# دليل رفع الموقع على Kubernetes

شرح كل خطوة عملناها لرفع ال Frontend على كلاسستر UAT: عملناها ليه، وإزاي، وإيه اللي حصل فيها، ومعها ال Pipeline ومصادر المذاكرة وموديول DevOps

## المحتويات

- |                                               |                                                |
|-----------------------------------------------|------------------------------------------------|
| 1. يعني إيه UAT؟                              | 11. الخطوة 7: تشغيل الموقع (Deployment)        |
| 2. يعني إيه Nexus؟                            | 12. الخطوة 8: فتح باب للموقع (Service)         |
| 3. الصورة الكبيرة                             | 13. شاشة Docker Desktop                        |
| 4. قاموس المصطلحات                            | 14. المشاكل اللي قابلتنا وحلّها                |
| 5. الخطوة 1: تجهيز الكود لبيئة UAT            | 15. اللي لسه ناقص                              |
| 6. الخطوة 2: بناء العلبة (Docker Image)       | 16. إزاي ترفع نسخة جديدة بعد ما تعدّل الكود    |
| 7. الخطوة 3: إعداد Docker Engine              | 17. ال Pipeline (CI/CD)                        |
| 8. الخطوة 4: رفع العلبة على Nexus             | 18. ال Pipeline الموجود عندنا ومشكلته          |
| 9. الخطوة 5: نسخ ملفات التعليمات للسيرفر      | 19. شكل ال Pipeline الصح (لما يبقى فيه Runner) |
| 10. الخطوة 6: باسورد Nexus للكلاسستر (Secret) | 20. تذاكر الحاجات دي منين                      |
|                                               | 21. موديول DevOps كامل                         |

---

التاريخ: 23 سبتمبر 2026 · الموقع شغال داخلياً على `http://10.225.151.51:30080`  
الباسوردات مش مكتوبة في الملف ده عشان الأمان. في الأوامر هتلاقي `REGISTRY_PASSWORD` و `MASTER_PASSWORD` ،  
وتكتب مكانهم الباسورد الحقيقي.

## 1. يعني إيه UAT؟

UAT = User Acceptance Testing، يعني "اختبار قبول المستخدم". هي نسخة تجريبية من الموقع بتتعمل قبل ما الموقع ينزل للناس الحقيقيين.

### الشركات عادةً عندها 3 نسخ من الموقع

| النسخة                                                                                           | مين بيستخدمها                         | الهدف                                      |
|--------------------------------------------------------------------------------------------------|---------------------------------------|--------------------------------------------|
| Development (على جهازك)                                                                          | إنت كمبرمج                            | تكتب الكود وتجرّبه وإنت شغال               |
| UAT  إحنا هنا | فريق الاختبار، والعميل، والـ business | يجربوا الموقع ويوافقوا عليه قبل ما ينزل    |
| Production                                                                                       | العملاء الحقيقيين                     | الموقع الفعلي، بفلوس حقيقية وبيانات حقيقية |

### تشبيه: مطعم جديد

- **Development**: الشيف بيجرّب الوصفة في مطبخه
- **UAT**: الافتتاح التجريبي، صاحب المطعم وأصحابه بيدوقوا الأكل ويقولوا رأيهم
- **Production**: الافتتاح الرسمي للناس كلها

### ليه UAT مهمة هنا بالذات؟

الموقع ده **Payment Aggregator**، يعني فلوس ومدفوعات. لو فيه غلطة ونزلت للناس على طول، ممكن فلوس حقيقية تضيع. فلازم يتجرّب الأول في مكان شبه الحقيقي بالظبط، بس ببيانات تجريبية.

### عشان كده الأسماء كلها فيها "uat"

| الاسم                                          | معناه                            |
|------------------------------------------------|----------------------------------|
| uatnoti.ultimate-pay.net                       | الموقع التجريبي                  |
| uat-apinoti.ultimate-pay.net                   | API التجريبي                     |
| environment.uat.ts و Dockerfile.uat و k8s/uat/ | ملفات خاصة بالنسخة التجريبية     |
| uat-payments namespace على الكلاستر            | المكان اللي فيه الـ API التجريبي |

لما UAT يتوافق عليها، هنعمل نفس الخطوات لـ Production بعناوين وملفات مختلفة (زي environment.prod.ts و k8s/prod/).

## 2. يعني إيه Nexus؟

**Nexus** (من شركة Sonatype) برنامج بيشتغل كـ **مخزن مركزي جوه الشركة**، بيتحط فيه أي حاجة الفرق محتاجة تشاركها مع بعض.

### تشبيه: مخزن الشركة

مخزن كبير في الشركة متقسم لأقسام، وأي موظف يحط فيه أو ياخذ منه. Nexus عندكم متقسم كده (كل قسم اسمه **repository**):

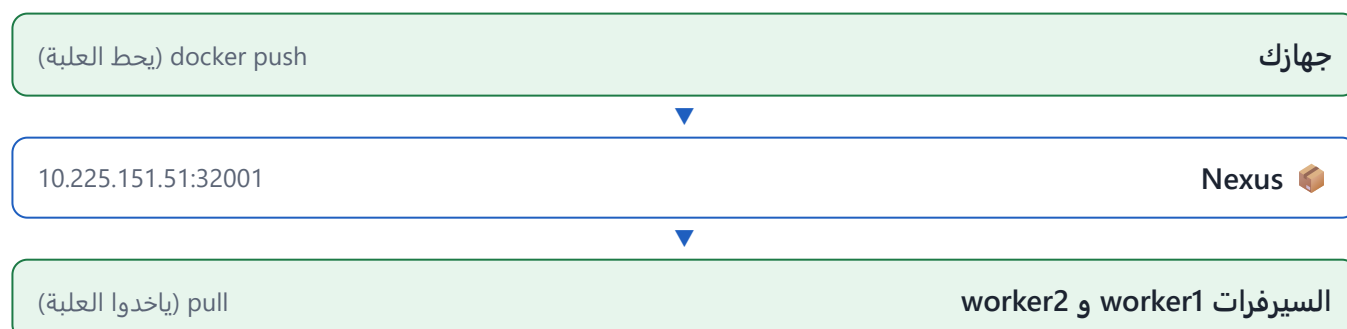
| القسم (repository) | بيتحط فيه إيه                | مين بيستخدمه |
|--------------------|------------------------------|--------------|
| maven-*            | مكتبات <b>Java</b>           | فرق Java     |
| nuget-*            | مكتبات <b>#NET / C.</b>      | فرق .NET     |
| docker ← ده بتاعنا | <b>Docker Images</b> (العلب) | إحنا         |

### أنواع الأقسام

| النوع  | معناه                                               | ينفع ترفع عليه؟       |
|--------|-----------------------------------------------------|-----------------------|
| hosted | مخزن بتاعكم، بتحطوا فيه حاجتكم                      | ✓ آه (وده نوع docker) |
| proxy  | نسخة محفوظة من مخزن على الإنترنت (زي maven-central) | ✗ لا                  |
| group  | بيجمع كذا مخزن تحت اسم واحد                         | ✗ لا                  |

### ليه محتاجينه؟

العلبة اتعملت على جهازك، والسيرفرات مش هتقدر تدخل على جهازك تاخدها. ف لازم مكان في النص الاتنين يوصلوله:



زي **Google Drive** بالظبط: إنت ترفع ملف وزميلك يحمله، من غير ما حد فيكم يدخل على جهاز الثاني.

ليه مش **Docker Hub** (المخزن العام على الإنترنت)؟

| Nexus               | Docker Hub          |                  |
|---------------------|---------------------|------------------|
| جوه شبكة الشركة     | على الإنترنت        | مكانه            |
| الشركة بس (بال VPN) | أي حد لو مش private | مين يشوفه        |
| أسرع (نفس الشبكة)   | أبطأ (من الإنترنت)  | السرعة للسيرفرات |

#### الأتنين مع بعض

عملنا نسخة **UAT** (تجريبية) من الموقع، وحطيناها في علبة، ورفعنا العلبة على **Nexus** (مخزن الشركة)، وسيرفرات الـ UAT خدتها من هناك وشغلّتها.

### 3. الصورة الكبيرة

تشبيه بسيط: مطعم 🍽️

| عندنا                                                             | في المطعم                                       |
|-------------------------------------------------------------------|-------------------------------------------------|
| عملنا build للموقع على جهازك                                      | طبخت الأكل في البيت                             |
| <b>Docker Image</b> : ملفات الموقع + السيرفر اللي بيعرضها (nginx) | حطيته في علبة مقفولة فيها الأكل والطبق والمعلقة |
| <b>docker tag</b> : إدينا ال image اسم فيه عنوان Nexus            | لزقت على العلبة ورقة بعنوان المخزن              |
| <b>docker push</b> على <b>Nexus</b>                               | وديت العلبة للمخزن                              |
| ملفات <b>YAML</b> ل <b>Kubernetes</b>                             | بعت للفروع ورقة: "خدوا العلبة دي وقدموها"       |
| <b>worker1</b> و <b>worker2</b> سحبوا ال image من Nexus           | الفرعين خدوا العلبة من المخزن                   |
| <b>Service</b> على بورت <b>30080</b>                              | فتحننا باب المطعم للناس                         |

#### رحلة الموقع

|                                                                    |                                     |
|--------------------------------------------------------------------|-------------------------------------|
| ... /src                                                           | كود Angular على جهازك               |
| ▼ <b>docker build</b> : بيعمل build جوه Docker ويحط الناتج في علبة |                                     |
| atlas-console:uat                                                  | Docker Image على جهازك              |
| ▼ <b>docker push</b> : بيرفع العلبة للمخزن                         |                                     |
| 10.225.151.51:32001                                                | Nexus (المخزن)                      |
| ▼ <b>Kubernetes</b> بي سحب العلبة (بالباسورد اللي في ال Secret)    |                                     |
| nginx بيعرض الموقع                                                 | 2 Pods شغالين على worker1 و worker2 |
| ▼ <b>Service</b> من نوع <b>NodePort</b> بيفتح بورت 30080           |                                     |
| http://10.225.151.51:30080                                         | الموقع شغال داخلياً ✓               |
| ▼ فريق ال infra يربط الدومين بالسيرفر (لسه)                        |                                     |
| https://uatnoti.ultimate-pay.net                                   | الموقع على الإنترنت 🕒               |

#### أهم فكرة في الدليل كله

إحنا مانسخرناش ملفات الموقع للسيرفر بإيدنا زي ما كنا بنعمل مع IIS. ملفات ال publish اتبنت جوه العلبة، والعلبة انتقلت بالكامل: جهازك ← Nexus ← السيرفرات. أمر scp نقل ورق التعليمات بس (ملفات YAML).

## 4. قاموس المصطلحات

| المصطلح                 | معناه ببساطة                                                                       |
|-------------------------|------------------------------------------------------------------------------------|
| <b>Docker</b>           | برنامج يحيط التطبيق وكل اللي محتاجه في علبة واحدة، تشتغل بنفس الشكل على أي جهاز    |
| <b>Image</b>            | العلبة المقفولة نفسها (ملف ثابت). فيها Linux صغير + nginx + ملفات الموقع           |
| <b>Container</b>        | العلبة وهي <b>شغالة</b> (ال image لما تشتغل)                                       |
| <b>Tag</b>              | اسم النسخة، زي <code>uat</code> أو <code>v1</code>                                 |
| <b>Dockerfile</b>       | الوصفة اللي بتقول لـ Docker يبنّي العلبة إزاي                                      |
| <b>nginx</b>            | سيرفر ويب خفيف بيعرض ملفات الموقع للمتصفح، زي IIS بالضبط                           |
| <b>Registry</b>         | مخزن الـ images، وعندنا هو <b>Nexus</b>                                            |
| <b>Nexus</b>            | برنامج مخازن في الشركة. فيه مخزن اسمه <code>docker</code> للـ images               |
| <b>Kubernetes (k8s)</b> | نظام بيشغل العلب على أكثر من سيرفر، ولو حاجة وقعت يشغلها ثاني لوحده                |
| <b>Cluster</b>          | مجموعة السيرفرات اللي Kubernetes بيديرها (عندنا 3)                                 |
| <b>Node</b>             | سيرفر واحد في الكلاستر: <b>master</b> (المدير) و <b>worker1 / worker2</b> (العمال) |
| <b>Pod</b>              | علبة شغالة جوه الكلاستر                                                            |
| <b>Deployment</b>       | أمر بيقول "عايز نسختين شغالين دايمًا"، ولو واحدة وقعت يعمل غيرها                   |
| <b>Service</b>          | عنوان ثابت يوصل للـ Pods (ال Pods عناوينها بتتغير)                                 |
| <b>NodePort</b>         | نوع Service بيفتح بورت على السيرفرات نفسها عشان تقدر توصل من برا الكلاستر          |
| <b>Secret</b>           | مكان آمن جوه الكلاستر لتخزين الباسوردات                                            |
| <b>Namespace</b>        | فولدر لتنظيم الحاجات جوه الكلاستر (إحنا في <code>default</code> )                  |
| <b>Ingress</b>          | راوتر بيوّج الترافيك حسب اسم الدومين. <b>مش متسطب عندكم</b>                        |
| <b>YAML</b>             | صيغة ملفات نصية بنكتب بيها التعليمات لـ Kubernetes                                 |
| <b>kubectl</b>          | الأداة اللي بنكلم بيها Kubernetes                                                  |
| <b>ssh</b>              | تدخل على سيرفر وتكتب أوامر عليه كأنك قاعد قدامه                                    |
| <b>scp</b>              | تنسخ ملفات من جهازك لسيرفر                                                         |
| <b>CORS</b>             | إذن لازم الـ API يديه للموقع عشان المتصفح يسمح للموقع يكلمه                        |

## 5. الخطوة 1: تجهيز الكود لبيئة UAT

مرة واحدة

على جهازك - في الكود

### المشكلة

الموقع لازم يعرف يكلم ال API فين (السيرفر اللي فيه ال login والبيانات)، والعنوان ده بيختلف حسب المكان:

| الموقع شغال فين       | بيكلم ال API على                     |
|-----------------------|--------------------------------------|
| على جهازك وإنت بتبرمج | /pa-api (عن طريق proxy)              |
| على IIS القديم        | /pa-api                              |
| على UAT (الجديد)      | https://uat-apinoti.ultimate-pay.net |

### الحل: 3 تعديلات

(أ) ملف إعدادات جديد: src/environments/environment.uat.ts

```
export const environment = {
  production: true,
  platformApiUrl: 'https://uat-apinoti.ultimate-pay.net',
  ...
};
```

معناه: "يا موقع، لما تحتاج بيانات، اطلبها من العنوان ده".

(ب) إعداد uat في angular.json

```
"uat": {
  "baseHref": "/",
  "fileReplacements": [
    { "replace": "src/environments/environment.ts",
      "with": "src/environments/environment.uat.ts" }
  ],
  "outputHashing": "all"
}
```

- **fileReplacements**: وقت ال Angular build، بيثيل ملف الإعدادات العادي ويحط مكانه ملف UAT.
- **baseHref**: "/" : الموقع يفتح على أول الدومين، مش جوه فولدر زي /UltimatePay/ بتاع IIS.
- **outputHashing**: أسماء الملفات بتأخذ كود مميز (زي main-a8f3c2.js)، فالمتصفح يحمل النسخة الجديدة بعد كل تحديث.

(ج) أمر مختصر في package.json

```
"build:uat": "ng build --configuration uat"
```

## النتيجة

أمر `npm run build:uat` بيطلّع ملفات الموقع الجاهزة (HTML و JS و CSS)، وهي نفس ملفات الـ `publish` اللي كنت بتنسخها لـ IIS، بس جواها عنوان الـ API الـ UAT.

## 6. الخطوة 2: بناء العلبه (Docker Image)

ال VPN مقفول (محتاج إنترنت)

على جهازك

### الأمر

```
docker build -f Dockerfile.uat -t atlas-console:uat .
```

- `-f Dockerfile.uat` : استخدم الوصفه دي (مش `Dockerfile` العادي بتاع IIS)
- `-t atlas-console:uat` : سمّي العلبه `atlas-console` ونسختها `uat`
- . (النقطه): الكود موجود في الفولدر الحالي

### الوصفه: ملف `Dockerfile.uat`

البناء بيحصل على مرحلتين: جهاز وهمي للبناء، وجهاز وهمي تاني للتشغيل.

```
# ---- المرحله 1: البناء ----
FROM node:22-alpine AS build
WORKDIR /app
COPY package.json package-lock.json ./
RUN npm ci
COPY . .
RUN npm run build:uat

# ---- المرحله 2: التشغيل ----
FROM nginx:1.27-alpine AS serve
COPY nginx.uat.conf /etc/nginx/conf.d/default.conf
COPY --from=build /app/dist/atlas-console/browser /usr/share/nginx/html
EXPOSE 80
```

| السطر                           | ييعمل إيه                                                                                               |
|---------------------------------|---------------------------------------------------------------------------------------------------------|
| FROM node:22-alpine AS build    | يجهّز جهاز وهمي صغير عليه Linux و Node.js (يبتحَمَل من Docker Hub على الإنترنت)                         |
| WORKDIR /app                    | يشتغل جوه فولدر /app                                                                                    |
| COPY package*.json + RUN npm ci | ينسخ ملفات ال packages ويسطّبها. عملناها لوحدها عشان Docker يحفظها في الكاش، فال build الثاني يبقى أسرع |
| COPY . .                        | ينسخ كود المشروع كله من جهازك لجوه الجهاز الوهمي                                                        |
| RUN npm run build:uat           | <b>هنا بيحصل ال publish</b> ، والملفات بتطلع جوه الجهاز الوهمي، مش على جهازك                            |
| FROM nginx:1.27-alpine          | يبدأ جهاز وهمي جديد ونضيف فيه nginx بس                                                                  |
| COPY nginx.uat.conf ...         | يحت ملف إعدادات nginx بتاعنا                                                                            |
| ★ COPY --from=build ...         | <b>ينقل ملفات ال publish</b> من جهاز البناء لفولدر nginx (/usr/share/nginx/html هو زي wwwroot في IIS)   |
| EXPOSE 80                       | توثيق إن nginx بيشتغل على بورت 80                                                                       |

### ليه مرحلتين؟

جهاز البناء (فيه Node.js و node\_modules) بيترمي في الآخر. العلبة النهائية فيها nginx وملفات الموقع بس، فحجمها صغير (حوالي 22MB بعد الضغط بدل مئات ال MB).

### ملف .dockerignore

فيه فولدرات Docker مش هينسخها ( node\_modules و dist و .git )، لأنها كبيرة ومالهش لازمة.

### ملف إعدادات nginx: nginx.uat.conf

```

server {
    listen 80;
    root /usr/share/nginx/html;
    index index.html;
    gzip on;

    location ~* \.(?:js|css|woff2?|svg|png|jpg|jpeg|ico)$ {
        expires 1y;
        add_header Cache-Control "public, immutable";
    }

    location / {
        try_files $uri $uri/ /index.html;
        add_header Cache-Control "no-cache";
    }
}

```

- **listen 80 / root**: اسمع على بورت 80، واعرض الملفات من الفولدر ده.
- **gzip on**: اضغط الملفات، فالموقع يحتمل أسرع.
- **ملفات js/css/الصور**: تتحفظ في كاش المتصفح لمدة سنة. ده آمن لأن أسماءها بتتغير مع كل build.
- **try\_files ... /index.html** ★: لو حد فتح `/dashboard` مباشرة أو عمل refresh، مفيش ملف بالاسم ده، ف nginx يرجع `index.html` و Angular يفتح الصفحة الصح. من غيره هيطلع 404.
- **no-cache على index.html**: المتصفح يسأل كل مرة على النسخة الجديدة، فبعد أي تحديث المستخدمين يشوفوه على طول.

## النتيجة

علبة اسمها `atlas-console:uat` فيها: Linux صغير + nginx + إعداداته + ملفات الموقع.

## 7. الخطوة 3: إعداد Docker Engine

مرة واحدة

على جهازك · Docker Desktop

### المشكلة

Docker من نفسه يرفض يرفع على أي مخزن شغال بـ `http` عادي، ولازم يكون `https` مشقّر. والـ Nexus بتاعكم شغال على `http` بس.

### اللي عملناه

Docker Engine ← Settings ← Docker Desktop، وضمنا السطر ده للـ JSON، وبعدين Apply & restart:

```
"insecure-registries": ["10.225.151.51:32001"]
```

معناه: "المخزن ده بالذات مسموحك تكلمه على HTTP".

### من غيره كان هيطلع الخطأ ده

```
http: server gave HTTP response to HTTPS client
```

### هل ده خطر؟

على شبكة داخلية وراها VPN مقبول، لأن الترافيك ماشي أصلاً جوه الـ VPN المشقّر. على الإنترنت العام ماينفعش.

## 8. الخطوة 4: رفع العلبة على Nexus

ال VPN شغال

على جهازك

### أ) نحت على العلبة ورقة بعنوان المخزن

```
docker tag atlas-console:uat 10.225.151.51:32001/docker/atlas-console:uat
```

ده مش بيعمل نسخة جديدة، ده بيدّي نفس العلبة اسم تاني. Docker بيعرف يرفع فين من الاسم نفسه:

| الجزء               | معناه                         | تشبيه   |
|---------------------|-------------------------------|---------|
| 10.225.151.51:32001 | عنوان Nexus (السيرفر والبورت) | المدينة |
| docker              | اسم المخزن جوه Nexus          | الشارع  |
| atlas-console       | اسم التطبيق                   | العمارة |
| uat                 | النسخة (tag)                  | الشقة   |

### ب) نسجل دخول في Nexus

```
docker login 10.225.151.51:32001 -u admin -p REGISTRY_PASSWORD
```

المفروض يطلع

Login Succeeded

### ج) نرفع

```
docker push 10.225.151.51:32001/docker/atlas-console:uat
```

هتشوف سطور زي Pushed 76b052d4ad30: دي الطبقات (layers)، لأن العلبة متقسمة لطبقات، وكل سطر في ال Dockerfile بيعمل طبقة. المرة الجاية هيرفع الطبقات اللي اتغيّرت بس.

المفروض يخلص بـ

uat: digest: sha256:e61e5f...

### تشوفها على Nexus إزاي؟

1. افتح http://10.225.151.51:32001 واعمل (admin Sign in)

2. من القائمة الشمال اختار Browse

3. ادخل على uat ← manifests ← atlas-console ← v2 ← docker

## 9. الخطوة 5: نسخ ملفات التعليمات للسيرفر

ال VPN شغال

من جهازك للسيرفر

### أ) ننسخ الملفات

```
scp -r k8s/uat master@10.225.151.51:~/atlas-console-uat
```

- scp : انسخ ملفات لسيرفر (Secure Copy)
  - -r : انسخ الفولدر باللي فيه
  - master@10.225.151.51 : اليوزر master على السيرفر ده
  - ~/atlas-console-uat : حطها في فولدر بالاسم ده على السيرفر
- هيطلب MASTER\_PASSWORD . ولو سأل Are you sure...? اكتب yes .

### انتسخ إيه بالضبط؟

3 ملفات نصية صغيرة بس: deployment.yaml و service.yaml و ingress.yaml . دي ورق تعليمات ل Kubernetes، ومافيهاش ولا ملف من ملفات الموقع.

### ب) ندخل على السيرفر

```
ssh master@10.225.151.51
```

بعد الباسورد، ال prompt هيتغير ل master@master:~\$ . كده إنت بتكتب أوامر على السيرفر نفسه.

### ج) نتأكد إن الكلاستر سليم

```
kubectl get nodes
```

|         |       |               |          |
|---------|-------|---------------|----------|
| master  | Ready | control-plane | ← المدير |
| worker1 | Ready | <none>        | ← عامل   |
| worker2 | Ready | <none>        | ← عامل   |

## 10. الخطوة 6: باسورد Nexus للكلاستر (Secret)

مرة واحدة

على السيرفر (master)

### المشكلة

ال `docker login` التي عملته كان على جهازك. ال workers سيرفرات ثانية، ومحتاجين يعملوا login في Nexus بنفسهم عشان يسحبوا العلبة.

### الأمر

```
kubectl create secret docker-registry uat-registry-creds \  
--docker-server=10.225.151.51:32001 \  
--docker-username=admin \  
--docker-password=REGISTRY_PASSWORD
```

### المفروض يطلع

```
secret/uat-registry-creds created
```

### ليه ماكتبناش الباسورد في ملف ال YAML؟

ملفات ال YAML بتتحط على Git وأي حد يشوفها. ال Secret بيتخزن جوه الكلاستر بس، وال YAML بيشاور عليه بالاسم ( uat-registry-creds ).

## 11. الخطوة 7: تشغيل الموقع (Deployment)

على السيرفر (master)

### الأمر

```
kubectl apply -f ~/atlas-console-uat/deployment.yaml -f ~/atlas-console-uat/service.yaml
```

**apply** معناه "خلي الكلاستر يبقى زي الملفات دي": لو الحاجة مش موجودة يعملها، ولو موجودة يحدّثها.

### ملف deployment.yaml : أهم الأجزاء

```
kind: Deployment
metadata:
  name: atlas-console-uat
spec:
  replicas: 2
  template:
    spec:
      containers:
        - name: atlas-console
          image: 10.225.151.51:32001/docker/atlas-console:uat
          imagePullPolicy: Always
          ports:
            - containerPort: 80
      resources:
        requests: { cpu: 50m, memory: 64Mi }
        limits: { cpu: 250m, memory: 128Mi }
        readinessProbe: { httpGet: { path: /, port: 80 } }
        livenessProbe: { httpGet: { path: /, port: 80 } }
      imagePullSecrets:
        - name: uat-registry-creds
```

| الجزء                   | معناه                                                                                                 |
|-------------------------|-------------------------------------------------------------------------------------------------------|
| replicas: 2             | نسختين شغالين دائماً. لو واحدة وقعت، الثانية شغالة، والتحديث بيبدلهم واحدة واحدة من غير ما الموقع يقف |
| image                   | العلبة اللي هتشتغل، ومن أنهى مخزن                                                                     |
| imagePullPolicy: Always | اسحب أحدث نسخة من Nexus كل مرة. <b>مهم</b> لأننا دائماً بنرفع بنفس الاسم uat                          |
| containerPort: 80       | nginx جوه العلبة بيستمع على 80                                                                        |
| resources               | الحد الأدنى والأقصى من ال CPU وال RAM، عشان الموقع مايستهلكش موارد السيرفر ويأثر على ال API           |
| readinessProbe          | كل 10 ثواني: "إنت جاهز؟" لو مردّش، مش هيبعتله زوار                                                    |
| livenessProbe           | كل 20 ثانية: "إنت عايش؟" لو مات، يعيد تشغيله لوحده                                                    |
| imagePullSecrets        | استخدم الباسورد اللي في ال Secret عشان تسحب من Nexus                                                  |

## اللي حصل أوتوماتيك بعد الأمر

1. ال master قرأ: "عايز نسختين من atlas-console"
2. قرّر: نسخة على worker1 ونسخة على worker2
3. كل worker عمل login في Nexus بال Secret وسحب العلبة
4. فتح العلبة وشغلها، وكده بقت Pod

## التأكد

```
kubectl get pods -l app=atlas-console
```

```
atlas-console-uat-5dcf459b8-ktdwj    1/1    Running
atlas-console-uat-5dcf459b8-rm9hm    1/1    Running
```

عايز تشوف ملفات الموقع جوه ال Pod بعينك؟

```
kubectl exec atlas-console-uat-5dcf459b8-ktdwj -- ls /usr/share/nginx/html
```

## 12. الخطوة 8: فتح باب للموقع (Service)

على السيرفر (master)

### المشكلة

الموقع اشتغل، بس ال Service كان من نوع **ClusterIP**، ودي معناها إن محدش يقدر يوصله غير من **جوه الكلاستر**.

### عرفنا الكلاستر بيفتح التطبيقات إزاي

```
kubectl get ingressclass → No resources found (Ingress مفيش)
kubectl get svc -A →
nexus-service NodePort 8081:32001
payment-aggregator-api NodePort 80:30746
```

كل التطبيقات عندهم مفتوحة بطريقة **NodePort**، فمشينا على نفس الطريقة.

| النوع     | مين يقدر يوصله                                              |
|-----------|-------------------------------------------------------------|
| ClusterIP | من جوه الكلاستر بس                                          |
| NodePort  | من برا، على السيرفر: البورت-IP (البورتات من 30000 لـ 32767) |

### الأمر

```
kubectl patch svc atlas-console-uat -p '{"spec":{"type":"NodePort","ports":[{"port":80,"targetPort":80,"nodePort":30080}]}}'
```

**patch** معناه "عدّل الحاجة دي وهي شغالة" من غير ما تمسحها. وعدّلنا `k8s/uat/service.yaml` في المشروع بنفس الشكل عشان الاتنين يفضلوا متطابقين:

```
spec:
  type: NodePort
  selector: { app: atlas-console, env: uat }
  ports:
    - port: 80          # جوه الكلاستر Service بورت الـ
      targetPort: 80    # جوه العلبة nginx بورت
      nodePort: 30080   # البورت المفتوح على السيرفرات
```

### النتيجة

الموقع فتح على `http://10.225.151.51:30080` ☒ (وال VPN شغال)

### ملف ingress.yaml

مش مستخدم لأن مفيش Ingress متسطب. سبيناه في المشروع لو ال infra سّطّبوا واحد بعدين.

## 13. شاشة Docker Desktop

في Docker Desktop ← Images هتلاقى 3 سطور. دي علبة واحدة بس عليها 3 ورقات (أسماء)، والدليل إن الـ Image ID واحد في الـ 3 ( e61e5f128668 )، ومكتوب فوق images 1 .

| الاسم                                    | جه منين                                                  | أعمل بيه إيه |
|------------------------------------------|----------------------------------------------------------|--------------|
| atlas-console                            | أول build، الاسم الأصلي المحلي                           | ممكّن تمسحه  |
| 10.225.151.51:32001/atlas-console        | أول محاولة رفع، والعنوان كان ناقصه docker<br>ففشلت (405) | ✗ امسحه      |
| 10.225.151.51:32001/docker/atlas-console | الاسم الصح اللي اترفع بيه                                | ✅ سيبه       |

### الأعمدة والأزرار

| الحاجة          | معناها                                            |
|-----------------|---------------------------------------------------|
| Tag             | اسم النسخة ( uat )                                |
| Image ID        | البصمة. لو متساوية يبقى هي نفس العلبة             |
| Size (76.83 MB) | الحجم على جهازك. وقت الرفع بيتضغظ لحوالي 22MB     |
| الدائرة الفاضية | مش شغالة على جهازك (طبيعي، هي شغالة على الكلاستر) |
| ▶               | تشغلها على جهازك للتجربة                          |
| 🗑               | تمسح الاسم ده بس. العلبة بتتمسح لما آخر اسم يتمسح |

### تمسح الاسم الغلط

```
docker rmi 10.225.151.51:32001/atlas-console:uat
```

هيطلع Untagged ، يعني شال الاسم بس. ده مش بيأثر على Nexus ولا على الموقع الشغال.

### تجربة: شغل الموقع على جهازك

```
docker run -d -p 8080:80 --name test-site atlas-console:uat
```

وافتح -p 8080:80 . http://localhost:8080 معناه إن بورت 8080 على جهازك يوصل لبورت 80 جوه العلبة. ولما تخلص امسحه:

```
docker rm -f test-site
```

### الـ Image موجودة فين دلوقتي؟

| المكان            | دوره           | لو اتمسحت منه                        |
|-------------------|----------------|--------------------------------------|
| جهازك             | مكان البناء    | مفیش مشكلة                           |
| <b>Nexus</b>      | المصدر الرئيسي | ⚠️ أي Pod جديد مش هيلاقى حاجة يسحبها |
| worker1 و worker2 | نسخة شغالة     | Kubernetes يسحبها ثاني لوحده         |

## 14. المشاكل اللي قابلتنا وحلّها

| المشكلة                                               | السبب                                                                         | الحل                                                                                |
|-------------------------------------------------------|-------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|
| The system cannot find the file specified             | كتبنا الباسورد بين < > ، والعلامة < في ال Command Prompt معناها "اقرأ من ملف" | شيل < > خالص. هي كانت علامة لمكان الكتابة بس                                        |
| no such host: registry-build 1.docker.io وقت ال build | ال VPN بيقطع الإنترنت، وال build محتاج يحمل node و nginx من Docker Hub        | ابني وال VPN مقفول، وارفع وال VPN شغال                                              |
| 405 HTTP method POST is not supported وقت ال push     | Nexus محتاج اسم المخزن في العنوان، وإحنا رفعنا على ...:32001/atlas-console    | العنوان الصح -docker/atlas:32001... console (عرفنا اسم المخزن من ال API بتاع Nexus) |
| kubectl get ingressclass ← No resources found         | مفیش Ingress متسطب على الكلاستر                                               | استخدمنا NodePort على 30080 زي باقي التطبيقات                                       |
| الدومين بيرجع 503                                     | الكلاستر على IP داخلي، و Cloudflare على الإنترنت مش واصل له                   | شغل فريق ال infra (الصفحة الجاية)                                                   |

### قاعدة ال VPN

**VPN مقفول:** فيه إنترنت، بس مفیش وصول للسيرفر ← **build**

**VPN شغال:** فيه وصول للسيرفر، بس مفیش إنترنت ← **push** و **ssh** و **kubectl**

ممکن تطلب من ال IT يعملوا **split tunneling**، يعني ال VPN ياخذ ترافيك الشبكة الداخلية بس والإنترنت يفضل شغال.

## 15. اللي لسه ناقص

### أ) ربط الدومين (فريق ال infra)

https://uatnoti.ultimate-pay.net بيرجع 503 من Cloudflare، لأن السيرفر على IP داخلي ( 10.225.151.51 ) و Cloudflare على الإنترنت مش واصل له.

| الدومين                               | المفروض يوصل لـ            |
|---------------------------------------|----------------------------|
| uatnoti.ultimate-pay.net (الموقع)     | http://10.225.151.51:30080 |
| uat-apinoti.ultimate-pay.net (API ال) | http://10.225.151.51:30746 |

الطرق الممكنة للربط: **Public IP / NAT** على الفايروول، أو **Cloudflare Tunnel**. ولو التوجيه HTTP عادي، إعداد ال SSL في Cloudflare لازم يبقى **Flexible**.

### ب) CORS (فريق ال Backend)

المتصفح بيعتبر uatnoti و uat-apinoti موقعين مختلفين، فبيمنع الموقع يكلم ال API إلا لو ال API سمح صراحة:

```
Access-Control-Allow-Origin: https://uatnoti.ultimate-pay.net
Access-Control-Allow-Credentials: true
```

Credentials لازمة لأن الموقع بيبعت ال cookie بتاعة ال login مع كل طلب. ومينفعش تبقى \* .

### ج) بعد ما يخلصوا

افتح https://uatnoti.ultimate-pay.net وال VPN مقفول:

- ✓ صفحة ال login ظهرت: الموقع اتربط
- ✗ لسه 503: الدومين لسه مش متوجّه
- ⚠ الصفحة ظهرت بس ال login مش شغال: ال API مش شغال أو ال CORS مش متطبط

## 16. إزاي ترفع نسخة جديدة بعد ما تعدّل الكود

الخطوات دي بس. الإعدادات اللي بتتعمل مرة واحدة (Docker Engine و Secret و Deployment و Service) مش هتعيدها.

### Build (1

VPN مقفول

على جهازك

```
cd /d "D:\Projects\Angular\Payment Agregator"
docker build -f Dockerfile.uat -t 10.225.151.51:32001/docker/atlas-console:uat .
```

### Push (2

VPN شغال

على جهازك

```
docker push 10.225.151.51:32001/docker/atlas-console:uat
```

### (3 ادخل السيرفر

VPN شغال

```
ssh master@10.225.151.51
```

### (4 خّلي الكلاستر يسحب النسخة الجديدة

على السيرفر

```
kubectl rollout restart deployment atlas-console-uat
kubectl get pods -l app=atlas-console
```

Kubernetes بيعمل Pods جديدة، ويسحب العلبة الجديدة (بسبب `imagePullPolicy: Always`)، ويبدّلهم واحدة واحدة، فالموقع مايقفش خالص.

### أوامر مفيدة للمتابعة

| الأمر                                                          | بيعمل إيه                                       |
|----------------------------------------------------------------|-------------------------------------------------|
| <code>kubectl get pods -l app=atlas-console -o wide</code>     | ال Pods شغالة فين وحالتها إيه                   |
| <code>kubectl describe pod POD_NAME</code>                     | تفاصيل Pod، ولو فيه مشكلة هتلاقها في Events تحت |
| <code>kubectl logs POD_NAME</code>                             | سجل nginx (مين فتح الموقع، وفيه أخطاء ولا لا)   |
| <code>kubectl rollout undo deployment atlas-console-uat</code> | ترجع للنسخة اللي قبلها لو الجديدة فيها مشكلة    |

ملفات المشروع اللي ليها علاقة

| الملف                               | وظيفته                          |
|-------------------------------------|---------------------------------|
| src/environments/environment.uat.ts | عنوان الـ API بتاع UAT          |
| angular.json ( إعداد uat )          | بيقول لـ Angular يستخدم ملف UAT |
| ( package.json ( build:uat )        | أمر البناء المختصر              |
| Dockerfile.uat                      | وصفة بناء اللعبة                |
| nginx.uat.conf                      | إعدادات nginx جوه اللعبة        |
| .dockerignore                       | فولدرات Docker مش هينسخها       |
| k8s/uat/deployment.yaml             | "شغل نسختين من اللعبة دي"       |
| k8s/uat/service.yaml                | "افتحهم على بورت 30080"         |
| k8s/uat/ingress.yaml                | مش مستخدم حالياً                |

# 17. ال Pipeline (CI/CD)

## يعني إيه Pipeline؟

كل اللي عملناه في الدليل ده (build ثم push ثم deploy) عملناه بإيدنا، أمر أمر. ال Pipeline هو إنك تخلي الكمبيوتر يعمل الخطوات دي لوحده أوتوماتيك، أول ما تعمل push للكود على GitHub.

## تشبيه: خط إنتاج في مصنع

بدل ما عامل يعمل كل حاجة بإيده، فيه سير متحرك: المنتج يدخل من أول الخط، ويعدي على محطات، ويطلع جاهز في الآخر. لو محطة لقت عيب، الخط يقف والمنتج البايز ما يوصلش للزبون.

## CI و CD

| الاسم                                 | معناه                                                                    | عندنا                                       |
|---------------------------------------|--------------------------------------------------------------------------|---------------------------------------------|
| CI = Continuous Integration           | كل ما حد يرفع كود، يتعمله build واختبار أوتوماتيك، عشان الغلط يتكشف بدري | build:uat ثم lint ثم npm ci ثم docker build |
| CD = Continuous Delivery / Deployment | النسخة اللي عدت الاختبارات تنتقل للسيرفر أوتوماتيك                       | docker push ثم تحديث Kubernetes             |

## شكل ال Pipeline بتاعنا



## المصطلحات

| المصطلح               | معناه                                                                                           |
|-----------------------|-------------------------------------------------------------------------------------------------|
| <b>GitHub Actions</b> | خدمة ال pipelines الموجودة جوه GitHub (فيه بدائل زي Jenkins و GitLab CI و Azure DevOps)         |
| <b>Workflow</b>       | ملف YAML فيه خطوات ال pipeline، مكانه <code>.github/workflows/</code>                           |
| <b>Trigger ( on :</b> | إمتى ال pipeline يشتغل: مع كل push، أو يدوي بزرار ( <code>workflow_dispatch</code> )            |
| <b>Job / Step</b>     | ال job مجموعة خطوات، وال step خطوة واحدة (أمر واحد)                                             |
| <b>★ Runner</b>       | الكمبيوتر اللي بينفذ الخطوات. ممكن يبقى جهاز من GitHub على الإنترنت، أو جهاز بتاعكم             |
| <b>Secrets</b>        | باسوردات متخزنة في (Settings ← Secrets) GitHub، وال pipeline بيستخدمها من غير ما تتكتب في الكود |

## 18. ال Pipeline الموجود عندنا ومشكلته

في المشروع ملف `.github/workflows/docker-uat.yml` ، اتعمل قبل ما نعرف تفاصيل الكلاستر:

```
name: Build and push UAT image
on:
  workflow_dispatch:           # Actions يشتغل يدوي بزرار من تاب
jobs:
  build-and-push:
    runs-on: ubuntu-latest      # على الإنترنت GitHub جهاز من Runner ال
    steps:
      - uses: actions/checkout@v4 # اسحب الكود
      - uses: docker/login-action@v3 # login في Docker Hub
        with:
          username: ${ secrets.DOCKERHUB_USERNAME }
          password: ${ secrets.DOCKERHUB_TOKEN }
      - uses: docker/build-push-action@v6 # build + push
        with:
          file: ./Dockerfile.uat
          push: true
          tags: ${ secrets.DOCKERHUB_USERNAME }/atlas-console:uat
```

### ليه ما استخدمنا هوش؟

1. بيرفع على **Docker Hub**، مش على Nexus.
2. الأهم: ال Runner بتاعه ( ubuntu-latest ) جهاز على الإنترنت عند **GitHub. Nexus** والكلاستر على IP داخلي ( 10.225.151.51 )، فالجهاز ده مش هيقدر يوصلهم خالص، نفس مشكلة Cloudflare.

### الحل: Self-hosted Runner

بدل ما GitHub يستخدم جهاز عنده، نسطب برنامج ال Runner على جهاز جوه شبكة الشركة. الجهاز ده بيعمل كده:

- بيكلم GitHub من جوه لبرا ويسأله "فيه شغل ليا؟"، فمش محتاج IP عام
- بيوصل ل Nexus والكلاستر لأنه على نفس الشبكة
- لازم يكون عنده إنترنت كمان (عشان GitHub و Docker Hub). يعني مايبقاش ورا VPN بيقطع الإنترنت

فيه الكود وال workflow

GitHub (على الإنترنت)

ال Runner بيسحب الشغل من GitHub ▼

مثلاً سيرفر على نفس شبكة الكلاستر

Self-hosted Runner (جوه الشركة)

push + kubectl ▼

10.225.151.51

Nexus + Kubernetes

### مين يستطبه؟

ده محتاج موافقة وجهاز من فريق ال infra. التسطيب من New ← Runners ← Actions ← Settings GitHub  
self-hosted runner. والجهاز لازم يكون عليه Docker و kubectl (ومعاه kubeconfig للكلاستر).

## 19. شكل ال Pipeline الصح (لما يبقى فيه Runner)

ده مثال لل workflow اللي هيحل محل الموجود. مش متفعل لسه، لأنه محتاج ال Runner الأول.

```
name: Deploy UAT

on:
  workflow_dispatch:      # زرار يدوي
  push:
    branches: [ uat ]      # اسم branch على push أو أوتوماتيك مع كل

env:
  IMAGE: 10.225.151.51:32001/docker/atlas-console

jobs:
  deploy:
    runs-on: self-hosted    # الجهاز اللي جوه الشركة
    steps:
      - name: Checkout
        uses: actions/checkout@v4

      - name: Build image
        run: docker build -f Dockerfile.uat -t $IMAGE:${{ github.sha }} -t $IMAGE:uat .

      - name: Login to Nexus
        run: echo "${{ secrets.NEXUS_PASSWORD }}" | docker login 10.225.151.51:32001 -u ${{ secrets.NEXUS_USERNAME }} --password-stdin

      - name: Push
        run: |
          docker push $IMAGE:${{ github.sha }}
          docker push $IMAGE:uat

      - name: Deploy to Kubernetes
        run: |
          kubectl set image deployment/atlas-console-uat atlas-console=$IMAGE:${{ github.sha }}
          kubectl rollout status deployment/atlas-console-uat --timeout=120s
    }}
```

| الخطوة               | بتعمل إيه                                                  | بدل إيه من اللي عملناه بإيدنا |
|----------------------|------------------------------------------------------------|-------------------------------|
| runs-on: self-hosted | ينفذ على جهاز الشركة                                       | جهازك + VPN                   |
| Checkout             | يسحب آخر كود                                               | الكود اللي على جهازك          |
| Build image          | بيني العلبة باسمين: uat ورقم ال commit                     | الخطوة 2 (docker build)       |
| Login to Nexus       | login بالباسورد من GitHub Secrets                          | docker login                  |
| Push                 | يرفع على Nexus                                             | الخطوة 4 (docker push)        |
| Deploy               | يقول لـ Kubernetes "استخدم النسخة دي"، ويستنى لحد ما تشتغل | ssh + kubectl rollout restart |

### ليه رقم ال (github.sha) commit في الاسم؟

كل نسخة بتأخذ اسم مختلف (زي atlas-console:8374d8e...)، فتعرف بالظبط أنه بي كود شغال على السيرفر، وتقدر ترجع لأي نسخة قديمة بأمر واحد. أما uat لوحده فبيكتب فوق نفسه كل مرة.

### المقارنة

|                   | يدوي (اللي عملناه)     | Pipeline                       |
|-------------------|------------------------|--------------------------------|
| مين بينفذ         | إنت، أمر أمر           | الكمبيوتر لوحده                |
| الوقت             | 10 لـ 15 دقيقة من شغلك | 0 دقيقة من شغلك                |
| VPN ال            | تقفل وتفتح             | مش محتاج                       |
| الأخطاء           | ممكن تنسى خطوة         | نفس الخطوات كل مرة             |
| مين رفع إيه وإمتى | مش متسجل               | متسجل في تاب Actions في GitHub |
| محتاج إيه         | جهازك + VPN            | Self-hosted Runner من ال infra |

### الخطوات لو حابب تعمله

- اطلب من ال infra جهاز جوه الشبكة عليه إنترنت، ويوصل لـ 10.225.151.51
- سّطب عليه Docker و kubectl، وخط عليه ال kubeconfig بتاع الكلاستر
- سّطب ال GitHub Runner عليه (Settings ← Actions ← Runners)
- ضيف NEXUS\_USERNAME و NEXUS\_PASSWORD في GitHub Secrets
- استبدل ملف docker-uat.yml بال workflow اللي فوق
- من تاب Actions في GitHub دوس Run workflow وتابع

## 20. تذاكر الحاجات دي منين

ذاكرهم بالترتيب ده، لأن كل حاجة مبنية على اللي قبلها:

|                           |                           |
|---------------------------|---------------------------|
| الأهم · حوالي أسبوع       | 1. Docker                 |
| ▼                         |                           |
| أصعب حاجة · حوالي أسبوعين | 2. Kubernetes             |
| ▼                         |                           |
| حوالي 3 أيام              | 3. CI/CD و GitHub Actions |
| ▼                         |                           |
| يوم واحد                  | 4. Nexus                  |

### Docker (1

| المصدر                                                                          | الرابط                                                                                          |
|---------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------|
| قناة <b>TechWorld with Nana</b> (دور على <i>Docker Tutorial for Beginners</i> ) | <a href="https://youtube.com/@TechWorldwithNana">youtube.com/@TechWorldwithNana</a>             |
| شرح Docker الرسمي للمبتدئين                                                     | <a href="https://docs.docker.com/get-started">docs.docker.com/get-started</a>                   |
| تجريب Docker في المتصفح (مجاني)                                                 | <a href="https://labs.play-with-docker.com">labs.play-with-docker.com</a>                       |
| مرجع أوامر الـ Dockerfile                                                       | <a href="https://docs.docker.com/reference/dockerfile">docs.docker.com/reference/dockerfile</a> |
| يوتيوب بالعربي                                                                  | بالعربي docker :بحث                                                                             |

لازم تفهم: image و container و Dockerfile و build و tag و push و pull و 8080:80 -p (ports).

### Kubernetes (2

| المصدر                                                        | الرابط                                                                                                              |
|---------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------|
| قناة Nana (دور على <i>Kubernetes Tutorial for Beginners</i> ) | <a href="https://youtube.com/@TechWorldwithNana">youtube.com/@TechWorldwithNana</a>                                 |
| شرح Kubernetes الرسمي (تفاعلي)                                | <a href="https://kubernetes.io/docs/tutorials/kubernetes-basics">kubernetes.io/docs/tutorials/kubernetes-basics</a> |
| ال Service وأنواعها (NodePort و ClusterIP)                    | <a href="https://kubernetes.io/.../service">kubernetes.io/.../service</a>                                           |
| ورقة أوامر kubectl المختصرة                                   | <a href="https://kubernetes.io/.../kubectl/quick-reference">kubernetes.io/.../kubectl/quick-reference</a>           |
| كلاستر مجاني في المتصفح للتدريب                               | <a href="https://killercode.com/playgrounds">killercode.com/playgrounds</a>                                         |
| كورسات ولابات عملية                                           | <a href="https://kodekloud.com">kodekloud.com</a>                                                                   |
| تفعيل Kubernetes على Docker Desktop (جهازك)                   | <a href="https://docs.docker.com/desktop/features/kubernetes">docs.docker.com/desktop/features/kubernetes</a>       |
| يوتيوب بالعربي                                                | بالعربي kubernetes: بحث                                                                                             |

**لازم تفهم:** Pod و Deployment و Service و Namespace و Secret و Ingress، وأوامر kubectl (get و describe و logs و apply و rollout).

### GitHub Actions (Pipeline) (3)

| المصدر                                | الرابط                                                                                                            |
|---------------------------------------|-------------------------------------------------------------------------------------------------------------------|
| البداية السريعة                       | <a href="https://docs.github.com/en/actions/quickstart">docs.github.com/en/actions/quickstart</a>                 |
| ال Self-hosted Runner (اللي محتاجينه) | <a href="https://docs.github.com/.../about-self-hosted-runners">docs.github.com/.../about-self-hosted-runners</a> |

### Nexus (4)

| المصدر                    | الرابط                                                                                          |
|---------------------------|-------------------------------------------------------------------------------------------------|
| Docker Registry ك Nexus   | <a href="https://help.sonatype.com/en/docker-registry">help.sonatype.com/en/docker-registry</a> |
| ال Nexus بتاعكم (بال VPN) | <a href="http://10.225.151.51:32001">http://10.225.151.51:32001</a>                             |

كتب 

[nigelpoulton.com/bo](https://nigelpoulton.com/bo) سهلين ومكتوبين للمبتدئين: *The Kubernetes Book* و **Nigel Poulton: Docker Deep Dive**   
 oks

طَبِّقْ عَلَى مشروعاتك 

| جرب على جهازك (مش على كلاستر الشركة)                                                                                                                                | بعد ما تذاكر          |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------|
| شغل الموقع بـ <code>docker run -d -p 8080:80 atlas-console:uat</code> وافتح <code>localhost:8080</code> . عدّل في <code>nginx.uat.conf</code> وابني تاني وشوف الفرق | <b>Docker</b>         |
| فعل Kubernetes في Docker Desktop، واعمل <code>kubectl apply</code> لملفات <code>k8s/uat/</code> عليه، وامسح pod وشوف Kubernetes بيرجّعه لوحده                       | <b>Kubernetes</b>     |
| اعمل repo تجريبي فيه workflow بسيط بيعمل <code>npm run build</code> مع كل <code>push</code>                                                                         | <b>GitHub Actions</b> |

### لو وقتك قليل

فيديو Docker وفيديو Kubernetes من قناة Nana (حوالي 4 ساعات)، وبعدهم جرب Kubernetes على Docker Desktop. ده هيخليك تفهم 80% من اللي في الدليل.

## 21. موديول DevOps كامل

اللي عملناه في الدليل ده جزء من **DevOps**، مش كله. ده الموديول الكامل، ومتعلّم فيه على اللي لمسته:

| #  | الموضوع                | يعني إيه باختصار                                         | عملته؟                     |
|----|------------------------|----------------------------------------------------------|----------------------------|
| 1  | Linux                  | نظام تشغيل السيرفرات وأوامره ( ls و cd و cat والصلاحيات) | ● شوية (ssh على ال master) |
| 2  | Networking             | IP و Ports و DNS و HTTP/HTTPS و Firewall و VPN           | ● شوية (VPN و 503 و CORS)  |
| 3  | Git متقدم              | Branches و Merge و Pull Requests                         | ● بتستخدمه                 |
| 4  | Docker                 | العلب                                                    | ✓ عملته                    |
| 5  | Container Registry     | Docker Hub و Nexus                                       | ✓ عملته                    |
| 6  | Kubernetes             | تشغيل العلب على سيرفرات                                  | ✓ الأساسيات                |
| 7  | CI/CD                  | ال Pipeline الأوتوماتيك                                  | ● فهمته، لسه ماعملتوش      |
| 8  | Helm                   | "مدير packages" ل Kubernetes، بدل ملفات YAML كتير        | ✗                          |
| 9  | Infrastructure as Code | تعمل السيرفرات نفسها بـكود (Terraform و Ansible)         | ✗                          |
| 10 | Cloud                  | AWS أو Azure أو GCP (سيرفرات مستأجرة)                    | ✗                          |
| 11 | Monitoring             | تعرف السيرفر تعبان ولا لأ (Prometheus و Grafana)         | ✗                          |
| 12 | Logging                | تجمع ال logs من كل ال pods في مكان واحد                  | ● ( kubectl logs بس)       |
| 13 | Security               | Secrets و HTTPS و Certificates والصلاحيات                | ● (ال Secret)              |
| 14 | Scripting              | Bash أو Python عشان تأتمت المهام                         | ✗                          |

📌 الترتيب المقترح

| المرحلة     | المواضيع                                                           | المدة تقريباً |
|-------------|--------------------------------------------------------------------|---------------|
| 1. الأساس   | Linux ثم Networking ثم Docker + Kubernetes (كمّل على اللي بدأت)    | شهر           |
| 2. الأتمتة  | CI/CD (GitHub Actions) ثم Helm ثم Bash scripting                   | شهر           |
| 3. الاحتراف | Terraform / Ansible ثم Cloud (واحد بس، AWS أو Azure) ثم Monitoring | شهرين أو أكثر |

## الروابط

| الموضوع                                                   | الرابط                                                                                                        |
|-----------------------------------------------------------|---------------------------------------------------------------------------------------------------------------|
| ★ خريطة DevOps التفاعلية: أهم رابط، فيها كل موضوع ومصادره | <a href="https://roadmap.sh/devops">roadmap.sh/devops</a>                                                     |
| Linux: لعبة بتعلّمك الأوامر مرحلة مرحلة                   | <a href="https://overthewire.org/wargames/bandit">overthewire.org/wargames/bandit</a>                         |
| Git: تعليم تفاعلي بالرسم                                  | <a href="https://learngitbranching.js.org">learngitbranching.js.org</a>                                       |
| Helm                                                      | <a href="https://helm.sh/docs">helm.sh/docs</a>                                                               |
| Terraform                                                 | <a href="https://developer.hashicorp.com/terraform/tutorials">developer.hashicorp.com/terraform/tutorials</a> |
| Ansible                                                   | <a href="https://docs.ansible.com">docs.ansible.com</a>                                                       |
| Prometheus (Monitoring)                                   | <a href="https://prometheus.io/docs/introduction/overview">prometheus.io/docs/introduction/overview</a>       |
| Grafana (Dashboards)                                      | <a href="https://grafana.com/tutorials">grafana.com/tutorials</a>                                             |
| TechWorld with Nana: عندها كورس DevOps كامل بنفس الترتيب  | <a href="https://youtube.com/@TechWorldwithNana">youtube.com/@TechWorldwithNana</a>                           |

## استغل اللي عندك 💡

عندك حاجة أغلب اللي بيتعلموا DevOps مش عندهم: **كلاستر حقيقي ومشروع حقيقي** في الشغل.

- لما تتعلم **Helm**: حوّل ملفات `k8s/uat/` لـ `Helm chart`
- لما تتعلم **CI/CD**: اعمل الـ Pipeline الحقيقي مع الـ `infra` (القسم 19)
- لما تتعلم **Monitoring**: اسأل الـ `infra` لو فيه Grafana عندهم، واعمّل dashboard للموقع وابدأ بـ [roadmap.sh/devops](https://roadmap.sh/devops)، وكل ما تخلص حاجة تعلّم عليها.